Securitatea in sisteme distribuite

Aspecte tratate in acest curs

Cursul trateaza:
- securitatea comunicatiei – intre utilizatori si/sau procese situate in calculatoare diferite
- securitatea accesului – un utilizator/proces utilizeaza doar serviciile/resursele la care are dreptul (autorizare) si
- gestiunea securitatii (cheilor, utilizatorilor, grupurilor).

Sistemele distribuite sunt mai vulnerabile datorita:
- controlului descentralizat
- accesului facut de la distanta
- tot mai multor aplicatii critice (Internet banking) instalate si executate ca sisteme distribuite.

Securitatea

Securitatea se refera la urmatoarele aspecte fundamentale:
- Confidentialitatea - informația este disponibilă doar utilizatorilor autorizați.
- Integritatea - informația poate fi modificată doar de utilizatorii autorizați sau în modalitatea autorizată (asigura ca mesajul primit nu a fost modificat în tranzit sau măsluit). Integritatea acopera atat **integritatea datelor**, cat si **integritatea originii** (verificata prin autentificare sau determinarea identității partenerului cu care schimbi mesaje înainte de a dezvălui informații importante).
- Disponibilitatea - accesul la informație a utilizatorilor autorizați nu este îngrădit (opusul este denial of service).

Dintre problemele derivate mentionam:
- controlul accesului, care inseamna protectia impotriva accesului ne-autorizat si
- non-repudierea, prin care se asigura ca transmitatorul nu poate nega transmiterea unui mesaj pe care un receptor l-a primit deja.

In tabelul alaturat mentionam cateva tipuri de amenintari si exemple de manifestare pentru cele doua tipuri de componente ale sistemelor distribuite: canale si obiecte (resurse).

Atac	**Canal**	**Obiect**
Inspectie	Citire continut mesaj	Citire date continute de obiect
Fabricare	Inserare mesaj	Parodiere obiect
Modificare	Schimbare mesaj	Schimbare date incapsulate
Intrerupere	Prevenirea transferului	Denial of service

Construire sistem sigur - etape

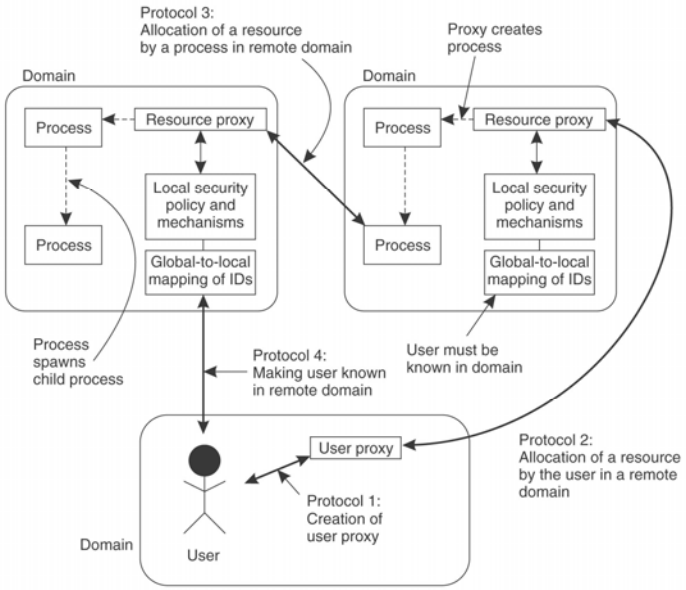
Construirea unui sistem sigur reclama parcurgerea etapelor descrise in continuare.

Etapa 1. Specificarea politicii de securitate (cerinte)
Politica precizeaza ce actiuni sunt permise unei entitati (care poate fi un utilizator, serviciu, o masina, un proces etc.)
Exemplu: Rezumam in continuare politica de securitate din Globus. Fiind un sistem distribuit pe scara larga, in Globus exista mai multe domenii administrative, fiecare domeniu avand propria politica locala de securitate. De regula, o aplicatie foloseste resurse aflate in mai multe domenii.

Pentru ca prelucrarile sa se deruleze fara probleme, este necesara o politica globala de securitate. Aceasta politica nu trebuie sa modifice, nici sa contrazica politicile locale. Regulile sale, prezentate pe scurt in continuare, se adauge politicilor locale si se refera exclusiv la operatii care afecteaza mai multe domenii:
- Operatiile locale unui singur domeniu se deruleaza conform politicii locale valabila pentru domeniul respectiv.
- Operatiile globale, care au efect in mai multe domenii, cer ca entitatea initiatoare sa fie cunoscuta in fiecare domeniu in care operatiile sunt executate. De exemplu, numele global al initiatorului este mapat pe nume locale, specifice domeniilor implicate.
- Operatiile inter-domenii cer autentificare mutuala. Daca un utilizator dintr-un domeniu apeleaza un serviciu din alt domeniu atunci utilizatorul se autentifica serviciului si serviciul, la randul sau, se autentifica utilizatorului, asigurandu-l ca foloseste serviciul dorit.
- Autentificarea globala tine loc si de autentificare locala. Daca un utilizator a fost verificat global atunci numele local pe care se mapeaza identificatorul global se considera de asemenea autentificat.
- Controlul accesului este supus securitatii locale. Dupa autentificarea unui utilizator, controlul accesului la resurse se face conform securitatii locale.
- Entitatile pot delega privilegii proceselor. Drept consecinta, dupa autentificarea unui proces caruia i-au fost delegate drepturi din partea proprietarului, verificarea privilegiilor acestuia se poate face fara a fi necesara contactarea proprietarului.
- Pentru procese din acelasi domeniu si actionand in contul aceluiasi utilizator, credentialele pot fi partajate in grup. Nu este deci necesar ca fiecare proces sa poarte setul sau unic de credentiale.

Etapa 2. Deoarece fiecare domeniu asigura securitatea propriei politici de securitate, **alegerea mecanismelor de securitate** (criptare, autentificare, autorizare) se concentreaza pe operatiile globale. De exemplu, pentru Globus, problemele critice sunt:
- Reprezentarea unui utilizator intr-un domeniu la distanta
- Alocarea resurselor dintr-un domeniu la distanta.

Etapa 3. Implementarea politicii globale de securitate este realizata prin protocoale.



Arhitectura sistemului (entitati si protocoale) este prezentata in figura. Entitatile sunt: utilizator, procese, user proxy si resource proxy. Un **user proxy** este un proces care actioneaza in numele

utilizatorului pentru un timp limitat. Un **resource proxy** este un proces folosit de user proxy pentru a transforma operatiile globale asupra resursei in operatii locale compatibile cu politica de securitate. Sunt utilizate patru protocoale, care reglementeaza urmatoarele operatii:
1 – creare proxy – si delegare de drepturi
2 – alocare resurse – se bazeaza pe crearea unui proces care reprezinta utilizatorul in domeniul distant; procesul are acces la resursa conform deciziilor locale de control al accesului
3 – alocare resurse - similar 2 dar intre proces si resource proxy
4 – mapare utilizator - utilizatorul are un cont intr-un domeniu; credentialele globale sunt convertite la credentiale recunoscute in acel domeniu

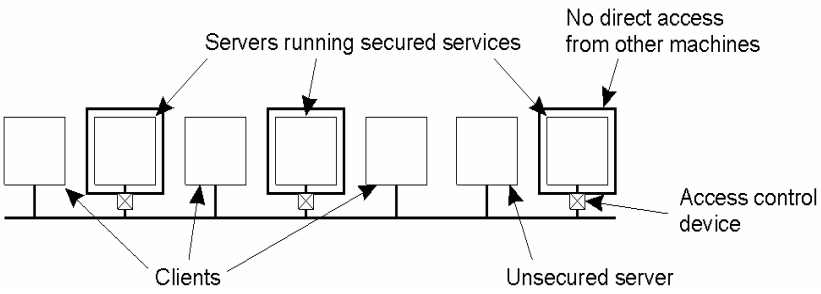
Prezentam principalii pasi pentru Protocolul 2 de alocare a resurselor.

Procesul creat reprezinta utilizatorul in domeniul distant; procesul are acces la resursa conform deciziilor locale de acces al controlului, conform unor credentiale (Cp) capatate prin acest protocolul:

1. user proxy - UP si resource proxy - RP se autentifica reciproc folosind certificate Cup si Crp. In acest pas, RP verifica daca credentialele UP nu au expirat.
2. UP prezinta catre RP o cerere de alocare semnata SigUP {specificatie alocare}.
3. RP verifica daca utilizatorul care a semnat credentialele UP este autorizat de politica locala sa faca cererea de alocare.
4. Daca cererea poate fi onorata, RP creaza un tuplu RESOURCE-CREDENTIALS continand numele utilizatorului caruia i se aloca resursa, numele resursei etc.
5. RP paseaza securizat RESOURCE-CREDENTIALS lui UP.
6. UP examineaza cererea RESOURCE-CREDENTIALS si, daca o aproba, semneaza tuplul producand Cp, credentialele pentru resursa ceruta.
7. UP paseaza securizat Cp lui RP.
8. RP aloca resursa si paseaza Cp noului proces. (acest transfer tine cont ca RP si procesul sunt in acelasi domeniu de incredere.)

Trusted Computer Base (TCB)

O componenta importanta in implementarea securitatii este TCB care identifica totalitatea componentelor si mecanismelor software si hardware cerute pentru a implementa o politica de securitate. De exemplu, daca sistemul distribuit este implementat ca middleware peste un sistem de operare de retea, TCB ar putea include sistemele de operare din diferitele calculatoare gazda. In figura este reprezentat principiul RISSC (Reduced Interfaces for Secure System Components) aplicat pentru securizarea sistemelor distribuite. Conform acestui principiu, serviciile sunt impartite in doua categorii, sigure si nesigure. Orice server cu cerinte critice de securitate este plasat pe o masina separata, izolata de sistemele utilizatorului. Clientii si aplicatiile pot accesa serverul doar prin interfete de retea sigure (echipamente de control al accesului de nivel scazut).



Etapa 4. *Asigurare* ca mecanismele functioneaza corect
Un pas important in implementarea sistemelor sigure este eliminarea sau reducerea vulnerabilitatilor si a efectelor lor. Pot fi aplicate mai multe metode, fiecare acoperind anumite cerinte ale asigurarii securitatii si avand propriile avantaje si dezavantaje.
Metodele formale sunt cele mai riguroase. Ele pot fi aplicate in toate etapele de realizare a securitaii: specificare, proiectare, implementare. Alte tehnici sunt cele de **testare** care pot evidentia existenta unor probleme dar nu pot demonstra absenta problemelor. Testele sunt mai usor de aplicat si ofera solutii mai rapide si mai putin costisitoare. In fine, **validarea** poate fi folosita pentru a asigura ca dezvoltatorii au implementat toate cerintele prin: inspectarea codului sursa, observarea comportarii la executie prin testare, inspectia documentatiei de proiectare sau alte metode.
Un aspect important este cel al **increderii** utilizatorilor in produsele folosite. Increderea reprezinta convingerea ca entitatea realizeaza politica de securitate pentru care a fost conceputa. Ea este o caracteristica generala legata de perceptia subiectului. Exista mai multe grade de incredere prin contrast cu securitatea care este o caracteristica binara a obiectului: acesta poate fi sigur sau nu. Deoarece evaluarea produselor trebuie sa se faca sistematic si organizat, au fost adoptate standarde care stabilesc criteriile si procedurile de evaluare, precum si clasele de incredere in care se pot incadra produsele evaluate. Un astfel de standard este ITSEC – Information Technology Security Evaluation Criteria.

Criterii de evaluare

Mentionam aici principalele standarde de evaluare si cateva informatii privind elaborarea lor.
TCSEC - Trusted Computer System Evaluation Criteria este un standard DoD cunoscut ca Orange Book (USA).
ITSEC - Information Technology Security Evaluation Criteria a fost publicat in 1990 de Franta, Germania, Olanda si UK.
Common Criteria - Common Criteria for Information Technology Security Evaluation este standardul ISO/IEC 15408 din 2005. El este derivat din ITSEC, TCSEC, CTCPEC (standard Canadian) si clasifica produsele IT dupa nivelul de incredere (Evaluation Assurance Level, EAL1 - EAL7) dupa cum urmeaza:

EAL1: Functionally Tested
EAL2: Structurally Tested
EAL3: Methodically Tested and Checked
EAL4: Methodically Designed, Tested, and Reviewed
EAL5: Semiformally Designed and Tested
EAL6: Semiformally Verified Design and Tested
EAL7: Formally Verified Design and Tested

(vezi: http://en.wikipedia.org/wiki/Evaluation_Assurance_Level)

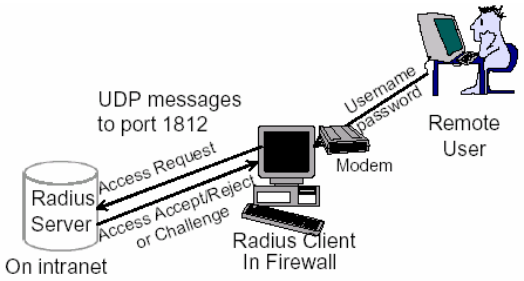
Autentificarea la distanta

Dupa cum s-a vazut din cele prezentate pana acum, autentificarea de la distanta a utilizatorilor sistemelor distribuite este un element important de securitate. Prezentam in continuare trei dintre cele mai cunoscute solutii de autentificare la distanta.

RADIUS - Remote Authentication Dial In User Service

Este un standard "de facto" pentru autentificarea la distanta. El cuprinde urmatorii pasi:
1. Utilizatorul intra in retea prin modem.

2. RADIUS client cere username si password.
3. RADIUS client trimite Access Request (codificat MD5).
4. RADIUS server autentifica utilizatorul si trimite Access Accept sau Access Reject la RADIUS client.



Pachetul RADIUS

In continuare, descriem succint protocolul RADIUS. Formatul pachetului este urmatorul:

```
 0                   1                   2                   3
 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|    Code       |   Identifier  |      Length                   |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|                                                               |
|                   Authenticator                               |
|                                                               |
|                                                               |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
| Attributes ...
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-
```

Campurile au urmatoarele semnificatii:

Code	Descriere	Code	Descriere
1	Access-Request	2	Access-Accept
3	Access-Reject	4	Accounting-Request
5	Accounting-Response	11	Access-Challenge
12	Status-Server (experimental)	13	Status-Client (experimental)
255	Reserved		

Actiuni client

Clientul RADIUS creaza un pachet **Access-Request** Campurile pachetului sunt generate astfel:
Identifier este generat de client si folosit pentru maparea raspunsului corespunzator de la server.
Request Authenticator, RA este un sir de 16 octeti alesi aleator.
Pachetul include cel putin atributele User-Name si User-Password.
Doar User-Password protejat, dupa algoritmul urmator:

Clientul si serverul partajeaza un secret S.
Password-ul este impartit in blocuri de 16-octeti p1, p2, ... pn (cu ultimul bloc completat la 16 octeti)
Blocurile de text cifrat sunt obtinute astfel:

c1 = p1 XOR MD5(S + RA)
c2 = p2 XOR MD5(S + c1)
...
cn = pn XOR MD5(S + cn-1)

Atributul User-Password contine c1+c2+...+cn
(+ denota concatenarea).

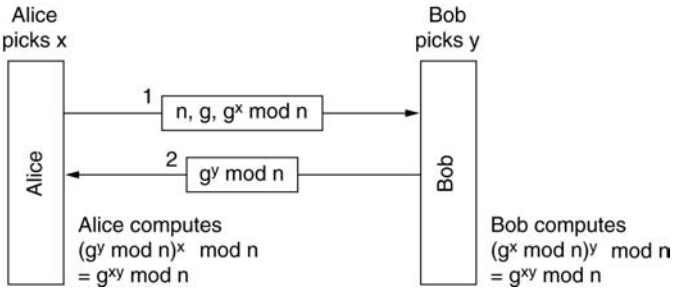
Actiuni server plus client

Server
La primirea unui pachet RADIUS Access-Request, serverul:
Verifica daca are un secret partajat cu clientul (daca nu, ignora pachetul primit).
Obtine password-ul neprotejat (procedura este similara celei urmate de client).
Foloseste Baza de Date de autentificare pentru a valida username si password.
Daca password valid => creaza pachet Access-Accept
Daca password invalid => creaza Access-Reject.
Adauga Response Authenticator in camp Authenticator. Acesta se calculeaza cu formula:
ResponseAuth = MD5(Code+ID+Length+RequestAuth+Attributes+Secret)
(+ denota concatenarea).

Client
Gaseste cererea corespunzatoare raspunsului (conform Identifier)
Calculeaza ResponseAuth (repetand operatii server) si compara rezultatul cu campul corespunzator din raspuns.
Ignora pachetul daca nu corespund.

Secure RPC

Protocolul a fost dezvoltat de Sun pentru sistemul de operare. Este bazat pe o combinatie de chei publice si chei secrete si anume:
DES pentru criptarea datelor
Diffie-Hellman pentru autentificare.
Reamintim modul de autentificare, care foloseste urmatoarele premise:
n, g – sunt numere mari, publice cunoscute, deci, si de Alice si Bob
x nu poate fi calculat din $g^x \bmod n$
g este un intreg cu proprietatea ca
orice numar p intre 1 si n-1 inclusiv poate fi scris ca $p = g^k \bmod n$.
n este prim si (n-1)/2 este prim.



Protocolul prin care Alice si Bob stabilesc o cheie secreta comuna are urmatoarele etape:
Alice alege un numar oarecare x (pe care-l tine secret) si transmite lui Bob mesajul 1. Secretul x nu poate fi aflat din $g^x \bmod n$. Bob alege un numar oarecare y (pe care-l tine secret) si transmite lui Alice mesajul 2. Secretul y nu poate fi aflat din $g^y \bmod n$. Bob calculeaza cheia secreta comuna $g^{xy} \bmod n$. Similar, Alice gaseste aceeasi cheie.

Secure RPC - caracteristici

Fiecare **principal** (care este un utilizator, calculator, proces autentificat) are o pereche de chei (Pastrate intr-o BD impreuna cu numele utilizatorului):
Publica, pastrata in clar

Secreta, pastrata in forma criptata DES cu password-ul sau (al principal-ului).
Un principal isi probeaza identitatea aratand ca poate decripta cheia secreta criptata, pe care o preia din BD.

Secure RPC – Local Login

U → C:	U, PW
C:	1. Retrieves from public key database a user record containing: username, user's public key, $\{\text{user's secret key}\}_{PW}$ 2. Decrypts secret key using PW and stores secret key in the keyserver process

Where:

U	user
PW	password

Apoi, el combina cheia secreta cu cheia publica a serverului, ajungand la o cheie de sesiune SK.
La randul sau, serverul preia cheia publica a clientului din BD si o combina cu cheia sa secreta obtinand aceeasi cheie de sesiune SK, comuna, calculata si de principal.
Clientul genereaza o cheie de conversatie (CK) si o transmite serverului in forma criptata cu cheia SK. Mesajul clientului mai include o fereastra de timp "window" si o amprenta de timp t1, ambele criptate cu CK. Fereastra indica diferenta maxima admisa intre ceasul serverului si amprenta de timp trimisa de server. Daca diferenta depaseste valoarea ferestrei atunci serverul ignora mesajul primit.
Serverul salveaza informatia impreuna cu un index ID folosit in continuare in conversatie si trimite clientului amprenta de timp t1 decrementata si criptata cu cheia de conversatie, impreuna cu indexul ID.

1. C:	1. Receives server's public key from public-key database. 2. Generates session key $(SK)_{c,s}$ for use between client and server.
2. C → S:	c, $\{CK\}_{SK(c,s)}$, $\{window\}_{CK}$, $\{t_1, window+1\}_{CK}$
3. S:	1. Receives client's public key from public-key database. 2. Generates session key $(SK)_{c,s}$ for use between client and server. 3. Decrypts the conversation key (CK) by using SK. 4. Decrypts the t_1, window and window+1. 5. Stores into a credential table, with an index (ID): C, CK, window, t_1.
4. S → C:	$\{t_1 - 1\}CK, ID$
5. C:	Stores ID and CK in key server process.

Where:

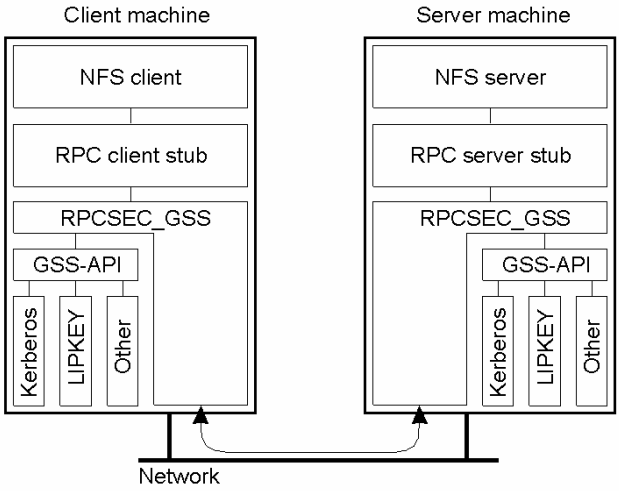
c	client
CK	conversation key
window	lifetime of CK
$SK_{c,s}$	secret key generated by client and server
t_1	original timestamp
t_n	current timestamp

In cererile ulterioare, clientul asociaza ID-ul si amprenta curenta de timp criptata cu CK, iar serverul asociaza raspunsului amprenta de timp decrementata si criptata cu CK si, bineinteles ID-ul. Cu acestea, clientul si serverul se asigura ca converseaza cu cine trebuie.

6. $C \rightarrow S$: $ID, \{t_n\}_{CK}$
7. $S \rightarrow C$: $\{t_n - 1\}_{CK}, ID$

Secure RPC in NFS versiunea 4

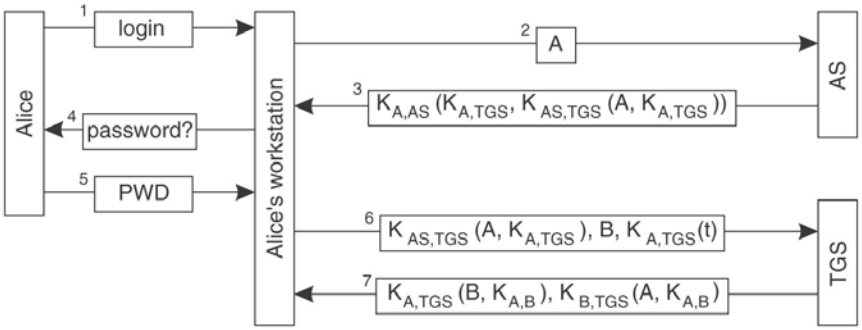
NFS este bazat pe RPC si foloseste atributele de securitate ale protocolului Secure RPC. Ceea ce ofera NSFv4 este un cadru general de securitate, RPCSEC_GSS – Generic Security Service, la realizarea caruia a stat o idee ingenioasa a dezvoltatorilor: in loc sa furnizeze propriile mecanisme de securitate, RPCSEC_GSS ofera posibilitatea de a incorpora diverse mecanisme de securitate cum ar fi Kerveros (care va fi prezentat in sectiunea urmatoare), LIPKEY - Low Infrastructure Public Key si altele. In particular, utilizatorilor sa se autentifice cu nume si parola si serverelor sa faca acelasi lucru folosind chei publice. Adaptarea la aceste mecanisme este facuta de o interfata standard pentru servicii de securitate GSS-API, rezultand arhitectura aratata in figura urmatoare.



Kerberos

Kerberos este sistemul de autentificare folosit în proiectul Athena la MIT. El presupune că stațiile de lucru sînt nesigure şi cere clientului să se identifice ori de cîte ori cere un serviciu. Pentru a simplifica procedeul folosit la autentificare, Kerberos foloseşte următoarele idei:
- utilizatorul trebuie să se identifice o singură dată, la începutul unei sesiuni (prin nume şi parolă);
- parolele nu sînt niciodată transmise prin rețea în clar, ci doar criptate; mai mult, ele nu sînt păstrate în clar în stațiile de lucru;
- fiecare utilizator are o parolă şi fiecare serviciu are o parolă;
- singura entitate care cunoaşte toate parolele este serverul de autentificare.

Schema de autentificare si autorizare este prezentata in figurile urmatoare in care sunt folosite notatiile:



AS – Authentication Server, serverul de autentificare
TGS – Ticket Granting Server, serverul de acordare a tichetelor
A – numele de login al lui Alice
password folosit pentru decriptarea mesajului 3.

Funcţionarea cuprinde următorii paşi:
(1) Utilizatorul deschide o sesiune şi transmite numele (ca răspuns la **login**).

(2) Staţia transmite serverului de autentificare AS un mesaj care conţine numele de login A. Mesajul nu este criptat, conţinînd nume ne-secrete.

(3) Serverul de autentificare (AS) caută în baza de date şi obţine:
- o cheie de criptare pentru utilizator, $K_{A,AS}$;
- o cheie de criptare pentru TGS, $K_{AS,TGS}$.
AS trimite un răspuns staţiei. Răspunsul conţine:
- un tichet care garantează accesul la SAT, $K_{AS,TGS}(A, K_{A,TGS})$
- o cheie de sesiune SAT, $K_{A,TGS}$.
Ambele sunt criptate cu $K_{A,AS}$.

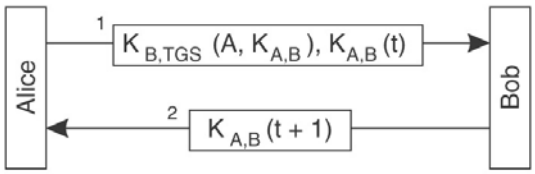
(4) Programul **login** primeşte mesajul şi cere parola utilizatorului.

(5) Parola este criptată printr-o funcţie greu inversabilă standard Unix, rezutatul fiind o cheie de cifrare a utilizatorului. Aceasta este folosită la decriptarea mesajului, parola clară fiind ştearsă din staţie. Staţia rămîne cu:
- un tichet sigilat;
- o cheie de sesiune TGS
pe care le memorează.

Folosirea unui serviciu reclamă obţinerea în prealabil a unui tichet corespunzător, de la SAT. Operaţiile următoare sînt executate automat şi transparent pentru utilizator.

(6) Staţia trimite TGS-ului un mesaj de forma (tichet_sigilat, nume serviciu B, amprenta de timp criptata cu cheia de sesiune TGS. TGS primeşte mesajul şi, mai întîi, decriptează tichetul sigilat, folosind cheia proprie de decriptare. De aici obţine cheia de sesiune TGS, cu care decriptează amprenta de timp pentru a stabili dacă mesajul este recent. TGS verifică numele din tichet (A). In fine, SAT obţine din baza de date cheia serviciului indicat, B.

(7) SAT formează o nouă cheie aleatoare de sesiune $K_{A,B}$ şi apoi creează un nou tichet in care include aceasta cheie si identitatea A, ambele criptate cu cheia secreta intre B si TGS, $K_{B,TGS}$. Totodata, identitatea serviciului B si cheia de sesiune $K_{A,B}$ sunt criptate cu cheia secreta intre A si TGS, $K_{A,TGS}$. Ambele valori criptare sun trimise statiei.



Cand utilizeaza serviciul B, statia trimite tichetul criptat cu $K_{B,TGS}$ si amprenta de timp criptata cu cheia secreta intre A si B, $K_{A,B}$ (mesajul 1). La returnarea rezultatului, B trimite si amprenta de timp modificata, criptata cu cheia secreta intre A si B (mesajul 2).